

Two-Stage Processing Pattern for System Design Interviews

 systemdesignschool.io/fundamentals/two-stage-processing



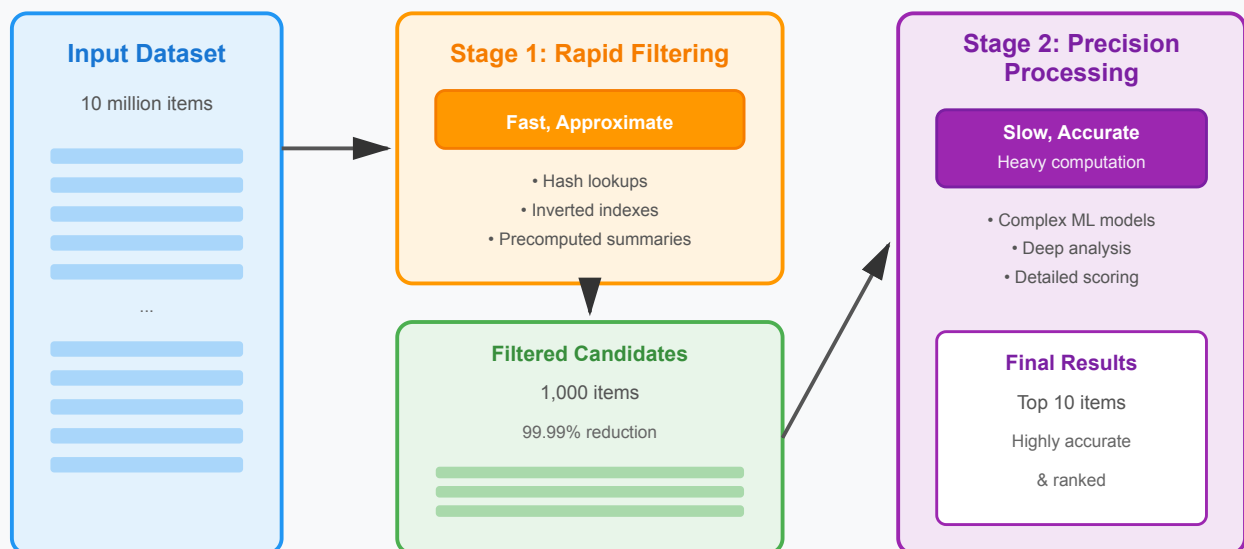
The Two-Stage Processing Pattern

A search engine indexes 10 million documents. When a user searches for "distributed systems", the system must return the 10 most relevant results ranked by quality. Running a complex relevance algorithm on 10 million documents takes too long. The search times out or the user waits seconds instead of milliseconds. Two-stage processing solves this by filtering data first then applying the expensive algorithm to a small set.

Fast Filter Then Precise Rank

Two-stage processing splits the work into rapid filtering and precise processing. The first stage quickly eliminates irrelevant candidates using simple, fast techniques. The second stage applies the expensive algorithm to the small remaining set.

Two-Stage Processing Pattern



Example: Search engine processes 10M documents → 1K keyword matches → Top 10 ranked results

Stage 1 (inverted index): 20ms | Stage 2 (ranking algorithm): 80ms | Total: 100ms vs 10+ seconds direct



Stage one prioritizes speed over precision. An inverted index maps keywords to documents. The query "distributed systems" looks up both words and intersects the document sets. This finds 50,000 documents containing both terms in milliseconds. The index uses hash lookups and boolean operations. No complex scoring happens yet.

Stage two prioritizes precision. The relevance algorithm examines each of the 50,000 candidates. It scores them using term frequency, document popularity, user location, and click-through history. This computation is expensive. Running it on 50,000 documents instead of 10 million makes it feasible. The system sorts by score and returns the top 10.

How This Scales

The filtering stage reduces the dataset by orders of magnitude. Processing 10 million items with a complex algorithm might take 30 seconds. Filtering to 1,000 candidates in 20 milliseconds then processing those in 80 milliseconds completes in 100 milliseconds total. The fast stage does most of the work. The slow stage runs on a tiny fraction of the data.

Each stage can scale independently. The filtering stage might run on machines with fast SSDs and lots of RAM for indexes. The precision stage might run on GPU machines for machine learning models. You can add more filtering machines to handle higher query rates. You can add more precision machines if scoring becomes the bottleneck.

Stage one can be approximate. A recommendation engine might use collaborative filtering to quickly find 500 similar users. Some of those users might not actually be similar. The second stage runs a deep learning model on those 500 profiles to predict which items to recommend. The deep model corrects mistakes from the fast filter.

Applications

Search engines use this pattern extensively. The inverted index finds keyword matches. The ranking algorithm scores relevance. Google processes billions of web pages this way. The index eliminates 99.99% of pages in milliseconds. Ranking runs on the remaining candidates.

Recommendation systems generate candidate items quickly then score them precisely. Netflix might use viewing history to find 1,000 potentially relevant movies in stage one. Stage two runs a neural network to predict ratings for those 1,000 movies and returns the top 10. Running the neural network on the entire catalog would be too slow.

Fraud detection flags suspicious patterns then investigates them deeply. Stage one checks transaction amounts, merchant categories, and geographic locations against simple rules. It flags 0.1% of transactions. Stage two runs machine learning models, checks external databases, and analyzes historical patterns for those flagged transactions.

Image similarity searches embed images into vectors then compares candidates. Stage one uses locality-sensitive hashing to find images with similar embeddings. This reduces 10 million images to 500 candidates. Stage two computes precise similarity scores using detailed feature comparisons.

Design Considerations

Stage one must not filter out correct results. If a search query filters out the most relevant document, stage two cannot recover it. The filtering stage should err on the side of including too many candidates rather than too few. Better to pass 10,000 items to stage two than to miss the best result by filtering too aggressively.

The intermediate data format matters. Passing 50,000 full documents from stage one to stage two wastes bandwidth. Pass document IDs instead. Stage two fetches the documents it needs. If stage two needs specific fields, include only those fields in the intermediate format.

Monitor the reduction ratio between stages. If stage one outputs 100,000 items and stage two struggles to process them quickly enough, tighten the filtering criteria. If stage one outputs 10 items and stage two is mostly idle, loosen the filtering to improve result quality.

Interview Application

When asked to design a search system, explain that ranking algorithms are too slow to run on every document. Propose a two-stage approach where an inverted index finds keyword matches quickly and a ranking algorithm scores those matches for relevance.

Discuss the trade-off between stage one accuracy and performance. A simpler filter runs faster but might miss relevant results or pass too many candidates to stage two. Show that you understand both stages must be tuned together.

Mention concrete techniques for each stage. Stage one might use hash lookups, inverted indexes, bloom filters, or locality-sensitive hashing. Stage two might use machine learning models, graph algorithms, or detailed similarity scoring. The choice depends on the specific problem.